RUNGTA INTERNATIONAL SKILLS UNIVERSITY

SCHOOL OF INFORMATION TECHNOLOGY
BACHELOR OF COMPUTER APPLICATION

PROJECT REPORT

PASSWORD STRENGTH CHECKER

Student Name	Prabhat Joshi
ERP ID	01191
Branch / Section	BCA 1B AIML
Project Number	PS No. 5
Programming Language	Python 3

Submitted in partial fulfillment of the academic project requirements.

1. Problem Statement

Build a Python program that checks a password against five security rules, classifies it as Weak, Medium, or Strong, and provides suggestions to improve it. The program must also reject empty passwords and passwords containing spaces.

2. Objectives

• Evaluate password strength using five security rules.

• Check password length, uppercase letters, lowercase letters, digits, and allowed special characters.

• Use Python regular expressions through the re module.

• Classify the password using conditional statements.

• Display suggestions for rules that are not satisfied.

• Handle empty input and spaces as invalid input.

3. Concepts Used

• **Strings:** Store and process the password.

• **Regular expressions (re):** Identify character patterns and spaces.

• **Conditions:** Classify the password based on its score.

• **Lists:** Store suggestions for failed rules.

• **Functions:** Organize the password-checking logic.

• **Sets:** The allowed special-character collection can be represented as a set; the code below uses a regular-expression character class for the required check.

4. Security Rules and Scoring

Rule	Requirement	Point
1	Length is at least 8 characters	1
2	Contains an uppercase letter (A–Z)	1
3	Contains a lowercase letter (a–z)	1
4	Contains a digit (0–9)	1
5	Contains one of @ # $ % & * !	1

Classification: 0–2 points = Weak; 3–4 points = Medium; 5 points = Strong.

5. Algorithm

1. Start the program and accept a password from the user.

2. If the password is empty, display “Password cannot be empty.” and stop checking.

3. If the password contains whitespace, display “Password must not contain spaces.” and stop checking.

4. Check each of the five security rules using length and regular expressions.

5. Add one point for every satisfied rule and record suggestions for failed rules.

6. Classify the score as Weak, Medium, or Strong.
7. Display the strength score and suggestions, then end the program.

6. Python 3 Source Code

```
import re

def check_password(password):
    # Check empty password
    if password == "":
        print("Password cannot be empty.")
        return

    # Reject spaces and other whitespace characters
    if re.search(r"\s", password):
        print("Password must not contain spaces.")
        return

    score = 0
    suggestions = []

    # Rule 1: Length must be at least 8 characters
    if len(password) >= 8:
        score += 1
    else:
        suggestions.append("use at least 8 characters")

    # Rule 2: At least one uppercase letter
    if re.search(r"[A-Z]", password):
        score += 1
    else:
        suggestions.append("add an uppercase letter")

    # Rule 3: At least one lowercase letter
    if re.search(r"[a-z]", password):
        score += 1
    else:
        suggestions.append("add a lowercase letter")

    # Rule 4: At least one digit
    if re.search(r"[0-9]", password):
        score += 1
    else:
        suggestions.append("add a digit")

    # Rule 5: At least one allowed special character
    if re.search(r"[@#$%&*!]", password):
        score += 1
    else:
        suggestions.append("add a special character")

    # Classify password strength
    if score <= 2:
        strength = "Weak"
    elif score <= 4:
        strength = "Medium"
    else:
        strength = "Strong"

    print(f"Strength: {strength} ({score}/5)")

    if suggestions:
        print("Suggestions: " + ", ".join(suggestions))
    else:
        print("No suggestions. Your password is secure.")


# Main program
password = input("Password: ")
check_password(password)
```

7. Test Cases and Expected Output

Test	Input	Expected output
1	abc123	Strength: Weak (2/5) Suggestions: use at least 8 characters, add an uppercase letter, add a special cha
2	Hello123	Strength: Medium (4/5) Suggestions: add a special character

Test	Input	Expected output
3(a)	(empty)	Password cannot be empty.
3(b)	Hello 123	Password must not contain spaces.
4	Hello@123Xy	Strength: Strong (5/5) No suggestions. Your password is secure.

8. How to Run the Program

• Install Python 3 on your computer.

• Save the source code in a file named password_strength_checker.py.

• Open a terminal in the folder where the file is saved.

• Run: **`python password_strength_checker.py`** (or **`py password_strength_checker.py`** on Windows).

• Enter a password when prompted and read the strength score and suggestions.

9. GitHub Submission

Create a GitHub repository named **Password-Strength-Checker**. Upload the Python file and this PDF report. Add a README.md file containing the project title, objective, requirements, and run instructions. Commit the files and submit the repository link as instructed by the faculty.

10. Conclusion

The Password Strength Checker evaluates a password against five security rules and classifies it as Weak, Medium, or Strong. It uses Python functions, strings, regular expressions, lists, and conditional statements. It also handles empty passwords and whitespace and provides suggestions to help users create stronger passwords.